ระบบลงทะเบียนที่แฮชรหัสผ่านโดยอัตโนมัติก่อนบันทึก และป้องกันรหัสผ่านหลุดผ่านทาง API

Function 5.1 - The Secure Registration (Bcrypt & Schema Logic)

📍 โจทย์

สร้างระบบสมัครสมาชิกที่ใช้ **Mongoose Pre-save Middleware** ในการจัดการความปลอดภัย โดยมีเงื่อนไขดังนี้:

1. **Auto-Hashing:** รหัสผ่านต้องถูกแฮชด้วย `bcrypt` (Salt Round = 12) ก่อนลง Database เสมอ
2. **Selective Selection:** ฟิลด์ `password` ต้องถูกตั้งค่าไม่ให้แสดงผล (hidden) เมื่อมีการ Query ข้อมูลทั่วไป
3. **Double-Save Prevention:** Middleware ต้องทำงานเฉพาะเมื่อรหัสผ่าน "มีการเปลี่ยนแปลง" เท่านั้น (ป้องกันการแฮชซ้ำซ้อนเมื่อ Update ฟิลด์อื่น)

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

🚀 Code

1. **`models/userModel.js`**
2. **`controllers/authController.js`**

🛠 Testing (Not Checklist)

1. **Setup:** ติดตั้ง `bcryptjs` , `mongoose` , และ `express`
2. **Implementation:** เขียนโค้ดในส่วนที่ขาดหายไปให้ครบถ้วน
3. **Verification 1 (The Hash):** ใช้ Postman ยิง `POST /api/register` จากนั้นไปตรวจสอบที่ **MongoDB Compass** ว่าค่าที่บันทึกเป็นค่าแฮช `$2a$12$...` หรือไม่
4. **Verification 2 (The Leak):** ตรวจสอบผลลัพธ์ (Response) จาก Postman ว่ามีฟิลด์ `password` หลุดออกมาหรือไม่ (ต้องไม่มี!)
5. **Verification 3 (The Logic):** ลองเขียนฟังก์ชัน Update ชื่อ (Name) ของผู้ใช้เดิม แล้วดูว่ารหัสผ่านใน Database เปลี่ยนแปลงหรือไม่ (รหัสผ่านต้อง "ไม่" ถูกแฮชซ้ำจนค่าเปลี่ยนไป)

⚠️ ข้อควรระวัง (The Trap)

- หากนิสิต ลืมตรวจสอบ `isModified('password')` ทุกครั้งที่มีการอัปเดตข้อมูลผู้ใช้ (เช่น เปลี่ยนชื่อ) Middleware จะทำการแฮชค่าแฮชเดิมซ้ำอีกครั้ง ทำให้ผู้ใช้เข้าสู่ระบบไม่ได้อีกเลย!

Checklist Function 5.1

C1. การแฮชรหัสผ่าน (Auto-Hashing Validation)

วิธีดู: ตรวจสอบผ่าน Database โดยตรง (MongoDB Compass / Atlas)

- **สิ่งที่ต้องเจอ:** ค่าในฟิลด์ `password` ต้องขึ้นต้นด้วยสัญลักษณ์ของ Bcrypt เช่น `$2a$12$...` หรือ `$2b$12$...`
- **จุดที่ต้องระวัง:** หากเห็นรหัสผ่านเป็นตัวอักษรปกติ (Plaintext) เช่น `123456` แสดงว่า Middleware `.pre('save')` ไม่ทำงาน หรือลืมสั่ง `await bcrypt.hash`
- **เทคนิคการเช็ค:** ความยาวของค่าแฮช Bcrypt จะมีความยาวคงที่คือ **60 ตัวอักษร** เสมอ

C2. การซ่อนข้อมูล (Selective Selection Validation)

วิธีดู: ตรวจสอบจาก JSON Response ใน Postman หรือผลลัพธ์จากคำสั่ง `User.find()`

- **สิ่งที่ต้องเจอ:** ใน JSON ที่ส่งกลับมาหา Client (Response Body) **ต้องไม่มี** ฟิลด์ `password` ปรากฏอยู่เลย
- **จุดที่ต้องระวัง:** หากคุณตั้งค่าใน Schema ว่า `select: false` แล้ว แต่ยังมีรหัสผ่านโผล่มา แสดงว่าคุณอาจจะไปเผลอใช้คำสั่ง `.select('+password')` ใน Controller หรือไม่ได้กำหนด `select: false` ไว้ในระดับ Schema จริงๆ

C3. การป้องกันการแฮชซ้ำ (Double-Hash Prevention)

วิธีดู: ทดสอบด้วยการอัปเดตฟิลด์ "อื่น" ที่ไม่ใช่รหัสผ่าน

- **ขั้นตอนการทดสอบ:**
  1. จดค่าแฮชรหัสผ่านเดิมในฐานข้อมูลไว้ (เช่น 5 ตัวท้าย)
  2. เขียนคำสั่งอัปเดต `email` หรือเพิ่มฟิลด์ `name` ของ User คนเดิม แล้วกด Save
  3. กลับไปดูค่าแฮชรหัสผ่านในฐานข้อมูลอีกครั้ง
- **สิ่งที่ต้องเจอ:** ค่าแฮชรหัสผ่าน **ต้องเหมือนเดิมเป๊ะ**
- **ถ้าไม่ผ่าน:** ค่าแฮชจะเปลี่ยนไป (เพราะโดนแฮชซ้ำ) ซึ่งจะทำให้ User คนนั้นล็อกอินไม่ได้อีกเลย เพราะรหัสผ่านเพี้ยนไปแล้ว

C4. การจัดการความแรงในการแฮช (Cost Factor Check)

วิธีดู: ตรวจสอบในโค้ดส่วน `bcrypt.hash(password, 12)`

- **สิ่งที่ต้องเจอ:** ตัวเลข Cost Factor (Salt Rounds) ควรเป็น **12** (ตามโจทย์)
- **ทำไมต้องเช็ค:** หากใส่ต่ำเกินไป (เช่น 4) จะแฮชเร็วมากแต่ไม่ปลอดภัย หากสูงเกินไป (เช่น 20) เซิร์ฟเวอร์จะค้างไปหลายวินาทีต่อการสมัคร 1 คน เลข 12 คือจุดที่สมดุลที่สุดในปัจจุบัน

C5. การป้องกัน Case Sensitive ใน Email (Data Normalization)

วิธีดู: ลองสมัครด้วย `User@Example.com` แล้วลองสมัครซ้ำด้วย `user@example.com`

- **สิ่งที่ต้องเจอ:** ระบบต้องไม่อนุญาตให้สมัครซ้ำ (Duplicate Key Error)
- **จุดที่ต้องเช็คในโค้ด:** ใน Schema ต้องมี `lowercase: true` เพื่อให้ระบบแปลงเป็นตัวพิมพ์เล็กทั้งหมดก่อนบันทึก ป้องกันการสับสนในการล็อกอิน

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

สร้างระบบ Login ที่ตรวจสอบรหัสผ่านอย่างปลอดภัย และส่งมอบ JWT ผ่านช่องทางที่ JavaScript ขโมยไม่ได้

Function 5.2 - The Double-Gate Login (JWT Issuance)

📍 โจทย์

สร้างระบบ Login ที่มีกลไกความปลอดภัย 2 ชั้น (Double-Gate) ดังนี้:

1. **Verification Gate:** สร้าง Instance Method ใน User Schema เพื่อเปรียบเทียบรหัสผ่านที่รับมา (Plaintext) กับรหัสผ่านใน DB (Hashed) โดยที่รหัสผ่านใน DB ต้องถูกดึงมาแบบ Explicit (เพราะเราทำ `select: false` ไว้ใน Function 5.1)
2. **Delivery Gate:** เมื่อตรวจสอบผ่าน ให้สร้าง JWT และส่งกลับไปหา Client ผ่าน **HttpOnly Cookie** เท่านั้น (ห้ามส่งใน JSON Body)
3. **Security Constraint:** JWT ต้องมีอายุสั้น (เช่น 15 นาที) และต้องมีการดึงค่า Secret Key จาก Environment Variable เท่านั้น

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

🚀 Code

1. `models/userModel.js` (เพิ่ม Instance Method)

2. `controllers/authController.js` (Login Logic)

⚠️ ระวังพลาด

- **The `this` context:** ใน Instance Method (`correctPassword`) หากคุณใช้ Arrow Function (`=>`) คุณจะไม่สามารถเข้าถึง `this.password` ได้ ต้องใช้ฟังก์ชันแบบธรรมดาเท่านั้น
- **Generic Error:** หากคุณเขียน Error แยกเช่น "User not found" กับ "Wrong password" คุณจะ **สอบตก** ในแง่ Security ทันที!

Checklist Function 5.2

C1. ตรวจสอบการดึงรหัสผ่าน (Manual Selection Check)

เหตุผล: เนื่องจากใน Function 5.1 เราตั้งค่า `select: false` ไว้ รหัสผ่านจะถูกซ่อนจากผลลัพธ์การ Query ทั่วไป

- **วิธีทดสอบ:** 1. ในไฟล์ `authController.js` ให้ลองเอาคำสั่ง `.select('+password')` ออกชั่วคราวแล้วกด Login

  2. **ถ้าผ่าน:** ระบบต้อง **Error** ทันที (เพราะ `user.password` จะเป็น `undefined` ทำให้ `bcrypt.compare` ทำงานไม่ได้)

  3. เมื่อใส่กลับเข้าไป ระบบต้องกลับมา Login ได้ปกติ
- **จุดที่ต้องระวัง:** อย่าลืมใส่เครื่องหมาย `+` หน้าฟิลด์ `password` ในคำสั่ง `.select()`

C2. ตรวจสอบตรรกะการเปรียบเทียบ (Bcrypt Comparison Check)

เหตุผล: เพื่อเช็คว่า Instance Method ใน Schema ทำงานถูกต้องร่วมกับ Bcrypt

- **วิธีทดสอบ:**
    1. ลอง Login ด้วยรหัสผ่านที่ **ผิด** (เช่น พิมพ์เกินไป 1 ตัว)
    2. **ถ้าผ่าน:** ระบบต้องตอบกลับด้วยรหัส **401 Unauthorized** พร้อมข้อความ Error
    3. **จุดสังเกตด้านความปลอดภัย:** ข้อความ Error ต้องเป็น **"Invalid email or password"** (ห้ามบอกว่า 'รหัสผ่านผิด' เพราะจะทำให้โจรเดาได้ว่าอีเมลนั้นมีอยู่ในระบบแล้ว)

C3. ตรวจสอบความปลอดภัยของ Cookie (HttpOnly Check)

เหตุผล: ป้องกันการโจมตีแบบ **XSS (Cross-Site Scripting)** ที่แฮกเกอร์ใช้ JavaScript มาขโมย Token

- **วิธีทดสอบ (ผ่าน Postman):**
    1. ยิง Request Login สำเร็จ
    2. คลิกที่แท็บ **"Cookies"** (อยู่ใต้ปุ่ม Send)
    3. ตรวจสอบคุกกี้ที่ชื่อ `jwt` : ต้องมีเครื่องหมายถูก (Tick) ในคอลัมน์ **HttpOnly**
- **วิธีทดสอบ (ผ่าน Browser Console):**
    1. ล็อกอินผ่านหน้าเว็บ (ถ้ามีหน้าบ้าน)
    2. เปิด Console (F12) แล้วพิมพ์ `document.cookie`
    3. **ถ้าผ่าน:** คุณต้อง **มองไม่เห็น** คุกกี้ที่ชื่อ `jwt` แม้ว่ามันจะถูกเก็บไว้ในเครื่องก็ตาม

C4. ตรวจสอบโครงสร้างของ JWT (JWT Payload Check)

เหตุผล: เพื่อดูว่าเราไม่ได้เผลอใส่ข้อมูลอันตรายลงไปในตั๋ว

- **วิธีทดสอบ:**
    1. คัดลอกค่า Token จากใน Cookie ไปวางที่เว็บไซต์ jwt.io
    2. ดูที่ส่วน **Payload (Data)**
    3. **ถ้าผ่าน:** ต้องมีแค่ `id` (ของ User), `iat` (เวลาที่ออก), และ `exp` (เวลาหมดอายุ)
    4. **ถ้าตก:** หากคุณเจอ `password` หรือ `email` หลุดอยู่ใน Payload แสดงว่าสอบตกเรื่องความเป็นส่วนตัวครับ

C5. ตรวจสอบการใช้ Secret Key (Environment Isolation)

เหตุผล: ป้องกันรหัสลับรั่วไหลเมื่อ Upload โค้ดขึ้น GitHub

- **วิธีทดสอบ:**
    1. ลองลบค่า `JWT_SECRET` ออกจากไฟล์ `.env` หรือปิดไฟล์ `.env` ไปเลย
    2. ลองรันเซิร์ฟเวอร์และ Login
    3. **ถ้าผ่าน:** ระบบต้อง **Crash** หรือ Error (เพราะ `process.env.JWT_SECRET` เป็น `undefined`)
    4. **จุดเช็ค:** ห้ามเขียน Key เป็น String ลงในโค้ด `jwt.sign(id, 'my-super-secret-key', ...)` เด็ดขาด

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

Middleware `protect` ที่ตรวจสอบสถานะปัจจุบันของผู้ใช้อย่างละเอียด ไม่ใช่แค่ตรวจสอบ Token

Function 5.3 - The Ghost Session (Security Middleware & Token Validation)

📍 โจทย์

แม้ว่า JWT จะเป็น Stateless แต่ต้องทำให้มัน กึ่ง Stateful เพื่อความปลอดภัยสูงสุด จงเขียน Middleware `protect` ที่ต้องผ่านด่านตรวจดังนี้:

1. **Token Existence:** ตรวจสอบว่ามี Token ส่งมาใน Header (`Authorization: Bearer ...`) หรือใน Cookie หรือไม่
2. **Verification:** ตรวจสอบความถูกต้องของ Signature และอายุ (Handling `TokenExpiredError`)
3. **User Existence (The Ghost Check):** แม้ Token จะถูก แต่ถ้า User เจ้าของ Token **ถูกลบออกจากระบบไปแล้ว** ต้องไม่อนุญาตให้เข้าถึงข้อมูล
4. **Password Change Check:** หาก User **เพิ่งเปลี่ยนรหัสผ่าน** หลังจากที่ Token นี้ถูกสร้างขึ้นมา Token ใบเดิมจะต้องใช้ไม่ได้ทันที (เพื่อป้องกันกรณี Token ถูกขโมยแล้วเจ้าของรีบเปลี่ยนรหัสผ่าน)

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

🚀 Code

1. `models/userModel.js` (เพิ่มฟิลด์และ Instance Method)

2. `middleware/authMiddleware.js` (The Protect Middleware)

⚠️ ระวังพลาด

- **The Date Format:** ฟิลด์ `passwordChangedAt` ใน MongoDB เก็บเป็น Milliseconds แต่ `iat` (Issued At) ใน JWT เก็บเป็น Seconds หากคุณเปรียบเทียบตรงๆ โดยไม่แปลงหน่วย **จะใช้การไม่ได้** เพราะ Logic จะผิดพลาดเสมอ
- **The `iat` vs `exp` :** ตรวจสอบให้ดีว่าคุณใช้ `iat` (เวลาที่ออก) ในการเช็ค ไม่ใช่ `exp` (เวลาที่หมดอายุ)

Checklist Function 5.3

C1. การทดสอบผู้ใช้ที่ไม่มีตัวตน (The Ghost User Test)

เป้าหมาย: ป้องกันกรณีพนักงานลาออกหรือโดนลบชื่อไปแล้ว แต่ยังมี Token เก่าเก็บไว้ในเครื่อง

- **วิธีทดสอบ:**
  1. ล็อกอินด้วย User A แล้วก๊อปปี้ Token (จาก Cookie หรือ Postman) เก็บไว้
  2. ไปที่ **MongoDB Compass** แล้วทำการ **Delete** User A ทิ้งไปเลย

3. นำ Token เดิมที่ก๊อปไว้ มาใช้เรียก API เส้นที่ติด `protect`

- **ถ้าผ่าน:** ระบบต้องตอบกลับ **401 Unauthorized** พร้อมข้อความประมาณว่า "User belonging to this token no longer exist"
- **ถ้าตก:** หากระบบยังให้เข้าถึงข้อมูลได้ (200 OK) แสดงว่า Middleware ของคุณเช็คแค่ความถูกต้องของ Token แต่ไม่ได้ไปดูใน Database ว่าเจ้าของตั๋วยังมีชีวิตอยู่ไหม

C2. การทดสอบการเปลี่ยนรหัสผ่าน (Password Change Test)

เป้าหมาย: เมื่อเจ้าของบัญชีเปลี่ยนรหัสผ่านใหม่ (เพราะสงสัยว่าโดนแฮก) Token ทุกใบที่เคยออกไปก่อนหน้านี้ต้องกลายเป็นเศษขยะทันที

- **วิธีทดสอบ:**
  1. ล็อกอินแล้วเก็บ Token ชุดที่ 1 ไว้
  2. ทำการเปลี่ยนรหัสผ่านของ User คนนี้ (หรือไปแก้ฟิลด์ `passwordChangedAt` ใน DB ให้เป็นเวลาปัจจุบัน)
  3. นำ Token ชุดที่ 1 (ใบเก่า) มาเรียก API อีกครั้ง
- **ถ้าผ่าน:** ระบบต้องตอบกลับ **401 Unauthorized** เพราะ Token ใบนี้ออกก่อนเวลาที่รหัสผ่านถูกเปลี่ยน
- **ถ้าตก:** หากยังเข้าได้ แสดงว่า Logic การเทียบเวลาใน `changedPasswordAfter` ของคุณผิดพลาด

C3. ตรวจสอบตรรกะเรื่องเวลา (Timestamp Logic Check)

เป้าหมาย: ป้องกันปัญหาเรื่อง "หน่วยเวลา" ที่ไม่เท่ากันระหว่าง DB (ms) และ JWT (s)

- **วิธีดูในโค้ด:**
  - ในเมธอด `changedPasswordAfter` คุณต้องมีการใช้ `parseInt(this.passwordChangedAt.getTime() / 1000, 10)` เพื่อเปลี่ยนจาก Milliseconds เป็น **Seconds**
- **ถ้าผ่าน:** ค่าทั้งสองฝั่งจะเป็นตัวเลขหลักหน่วยวินาที (เช่น 1614523000) เท่ากัน
- **ถ้าตก:** หากคุณเอา `Date` มาเทียบกับ `iat` ตรงๆ โดยไม่หาร 1000 ค่าจะห่างกันเป็นพันเท่า และ Logic จะเพี้ยนทันที (ทำให้เข้าไม่ได้เลย หรือเข้าได้ตลอดเวลา)

C4. การฉีดข้อมูลผู้ใช้ (Decoded Payload Injection Check)

เป้าหมาย: เพื่อให้ Controller ตัวถัดไป (เช่น `updateMe` หรือ `deleteMe`) รู้ว่าใครเป็นคนส่ง Request มา

- **วิธีดูในโค้ด:** หลังจากด่านตรวจทุกอย่างผ่าน ใน Middleware ต้องมีคำสั่ง `req.user = currentUser;`
- **วิธีทดสอบ:**
  1. ใน Controller ตัวถัดไป ให้ลอง `console.log(req.user)`
- **ถ้าผ่าน:** คุณต้องเห็น Object ข้อมูลของ User (ที่ดึงมาจาก DB) แสดงใน Terminal
- **ถ้าตก:** หาก `req.user` เป็น `undefined` แสดงว่า Middleware ตรวจเสร็จแล้วแต่ไม่ได้ "ส่งของ" ต่อให้เพื่อน

C5. การดักจับ Error (Error Handling Check)

เป้าหมาย: เพื่อไม่ให้เซิร์ฟเวอร์ค้าง (Crash) เมื่อแฮกเกอร์ส่ง Token มั่วๆ มาทดสอบ

- **วิธีทดสอบ:**
  1. ลองแก้ตัวอักษรใน Token เพียง 1 ตัว (เช่น จาก 'a' เป็น 'b') แล้วยิง Request
- **ถ้าผ่าน:** ระบบต้องตอบกลับ **401** และแสดงข้อความว่า "Invalid token" (มาจากบล็อก `catch`)
- **ถ้าตก:** เซิร์ฟเวอร์จะค้าง หรือส่ง Error 500 กลับไป ซึ่งแสดงว่า Middleware จัดการ Error ได้ไม่ดีพอ

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

Middleware ที่สามารถจำกัดสิทธิ์การเข้าถึงตามบทบาท (Roles) โดยใช้แนวคิด Factory Function เพื่อความยืดหยุ่นสูงสุด

Function 5.4 - Dynamic Role-Based Access Control (RBAC)

📍 โจทย์

เปลี่ยนจากแค่การเช็ค "ว่าเป็นใคร" มาเป็นการเช็คว่า "ใหญ่แค่ไหน" ในระบบ

สร้างระบบจัดการสิทธิ์ที่สามารถ "กั้นเขต" ให้กับแต่ละ Route ได้ โดยมีเงื่อนไขดังนี้:

1. **Role Factory:** สร้างฟังก์ชัน `restrictTo` ที่รับ Parameter เป็นชื่อ Role ได้หลายชื่อ (เช่น `'admin'`, `'editor'`)
2. **Authorization Check:** Middleware นี้ต้องตรวจสอบค่า `role` จาก `req.user` (ที่ได้มาจาก `protect` ใน Function 5.3)
3. **The Hierarchy Logic (Hard Mode):** จงเขียน Logic ให้ระบบรองรับลำดับชั้น (Hierarchy) โดยกำหนดให้ Role **`admin`** สามารถเข้าถึง Route ของ **`editor`** และ **`user`** ได้เสมอ แม้ในคำสั่ง `restrictTo` จะไม่ได้ระบุชื่อ `admin` ไว้ก็ตาม
4. **Error Protocol:** หากผู้ใช้ไม่มีสิทธิ์ (สิทธิ์ไม่ถึง) ต้องตอบกลับด้วยรหัส **403 Forbidden** (ไม่ใช่ 401)

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

🚀 Code Starter Template

1. `models/userModel.js` (เพิ่มฟิลด์ Role)

2. `middleware/authMiddleware.js` (The Restrict Middleware)

⚠️ ระวังพลาด

- **The `req.user` Missing:** หากนิสิตรัน `restrictTo` ก่อน `protect` ตัวแปร `req.user` จะเป็น `undefined` และระบบจะพังทันที (Middleware นี้ต้องทำงานเป็นลำดับ)
- **Enum Integrity:** หากนิสิตพิมพ์ชื่อ Role ในโค้ดผิด (เช่น `'Admin'` ตัวใหญ่ แต่ใน DB เก็บ `'admin'` ตัวเล็ก) Logic จะล้มเหลวทันที แนะนำให้ใช้ตัวพิมพ์เล็กทั้งหมด

📋 คำแนะนำ (ไม่ใช่ Check List)

1. **ดูรหัส HTTP:** * หากหน้าจอขึ้น **403 Forbidden** เมื่อใช้ ID ทั่วไปเข้าหน้า Admin = **ผ่าน**
   - หากขึ้น **401 Unauthorized** = **ตก** (เพราะแสดงว่าคุณแยกความแตกต่างระหว่าง "ไม่รู้ว่าเป็นใคร" กับ "ไม่มีสิทธิ์" ไม่เป็น)
2. **ดูความใจกว้างของ Admin:**
   - หากคุณสั่ง `restrictTo('editor')` แต่ Admin เข้าไม่ได้ = **ตก** (เพราะ Logic Hierarchy ของคุณยังไม่ทำงาน)
3. **ดูตำแหน่งใน Router:**
   - `router.delete('/user/:id', auth.protect, auth.restrictTo('admin'), userController.deleteUser)`
   - หากสลับตำแหน่ง `restrictTo` มาไว้หน้า `protect` = **ตก** (ระบบจะ Error ทันทีที่ยิง Request)

สิ่งที่ยากที่สุดไม่ใช่การเขียนโค้ด แต่คือการจัดการ **"ตรรกะลำดับชั้น" (Logic Hierarchy)** เพื่อไม่ให้ระบบเกิดอาการ "แอดมินเข้าหน้าตัวเองไม่ได้"

Checklist Function 5.4

C1. การทดสอบการปฏิเสธสิทธิ์ (The Forbidden Test)

เป้าหมาย: เพื่อดูว่า Middleware สามารถกั้นคนที่ไม่เกี่ยวข้องออกไปได้จริงไหม

- **วิธีทดสอบ:** 1. สร้าง Route ใหม่ เช่น `GET /api/editor-only` โดยใส่ Middleware `restrictTo('editor')`

  2. ล็อกอินด้วย User ที่มี `role: 'user'` แล้วนำ Token มาเรียก Route นี้
- **ถ้าผ่าน:** ระบบต้องตอบกลับ **403 Forbidden** เท่านั้น
- **ถ้าตก:** หากระบบให้เข้า (200 OK) แสดงว่า Logic การเช็ค Array ของคุณผิด หรือหากระบบตอบ 401 แสดงว่าคุณสับสนระหว่างการเช็คตัวตน (AuthN) กับการเช็คสิทธิ์ (AuthZ)

C2. การทดสอบลำดับชั้น (The Hierarchy Test)

เป้าหมาย: พิสูจน์ว่า `admin` คือพระเจ้าของระบบ เข้าได้ทุกที่โดยไม่ต้องระบุชื่อในทุก Route

- **วิธีทดสอบ:**
  1. ใช้ Route เดิมจากข้อ 1 (`restrictTo('editor')`)
  2. ล็อกอินด้วย User ที่มี `role: 'admin'` แล้วนำ Token มาเรียก
- **ถ้าผ่าน:** ต้องได้ **200 OK** (แม้เราจะไม่ได้เขียนคำว่า 'admin' ลงใน `restrictTo` ของ Route นั้นก็ตาม)
- **วิธีดูในโค้ด:** คุณต้องมีคำสั่ง `if (req.user.role === 'admin') return next();` ไว้ที่บรรทัดแรกๆ ของ Middleware

C3. การทดสอบหลายบทบาท (Multiple Roles Test)

เป้าหมาย: เช็คว่าระบบรองรับการระบุหลายสิทธิ์พร้อมกันได้ไหม (เช่น หน้าที่ให้ทั้ง Editor และ Manager เข้าได้)

- **วิธีทดสอบ:**
  1. ตั้งค่า Route เป็น `restrictTo('editor', 'manager')`
  2. ทดสอบล็อกอินด้วย ID ที่เป็น `editor` -> ต้องเข้าได้
  3. ทดสอบล็อกอินด้วย ID ที่เป็น `manager` -> ต้องเข้าได้
  4. ทดสอบด้วย `user` -> ต้องโดนเตะออก (403)
- **วิธีดูในโค้ด:** คุณควรใช้คำสั่ง `allowedRoles.includes(req.user.role)` ในการตรวจสอบ

C4. การตรวจสอบรหัสสถานะ (Status Code Check)

เป้าหมาย: ความเป็นมืออาชีพในการสื่อสารกับ Frontend

- **วิธีทดสอบ:** ดูผลลัพธ์ใน Postman ตอนที่ถูกปฏิเสธสิทธิ์
- **ถ้าผ่าน:** ต้องเห็นเลข **403** ชัดเจน
- **ทำไมต้อง 403?:** เพราะถ้าส่ง 401 Frontend จะนึกว่า Token หมดอายุแล้วจะเตะ User กลับไปหน้า Login ทันที ทั้งที่จริงๆ เขาแค่ไม่มีสิทธิ์เข้าโซนนั้นเฉยๆ (User ควรจะยังอยู่ในระบบได้ต่อ)

C5. ลำดับของ Middleware (Middleware Order Check)

เป้าหมาย: ป้องกันระบบพัง (Runtime Error)

- **วิธีดูในโค้ด:** เปิดไฟล์ `routes.js` แล้วดูการเรียงลำดับ
- **ถ้าผ่าน:** ต้องเห็นโครงสร้างแบบนี้:

  `router.get('/admin-zone', auth.protect, auth.restrictTo('admin'), controller.fn)`
- **ถ้าตก:** หากเอา `restrictTo` ไว้ก่อน `protect` -> ระบบจะหา `req.user` ไม่เจอ และจะ Error `Cannot read property 'role' of undefined` ทันทีที่เรียก API

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

สร้างเกราะป้องกันการเดารหัสผ่าน (Brute Force) และระบบ Logout ที่ปลอดภัยจริงโดยการทำ Token Blacklisting

Function 5.5 - Secure Token Rotation & Anti-Brute Force Layer

---

📍 โจทย์ (The Challenge)

จงยกระดับระบบรักษาความปลอดภัยให้สมบูรณ์ด้วย 3 กลไกสำคัญ:

เข้าได้หรือไม่ ยังไม่พอแค่นั้น แต่ต้องเน้นที่การ **"ป้องกันการโจมตี"** และการ **"ปิดช่องโหว่"** ที่แฮกเกอร์มักใช้โจมตีระบบจริง

1. **Rate Limiting:** จำกัดการพยายาม Login ของแต่ละ IP ไม่ให้เกิน 5 ครั้ง ภายใน 15 นาที เพื่อป้องกันการใช้ Script สุ่มรหัสผ่าน
2. **Login Delay (Artificial Latency):** หาก Login ผิด ระบบต้อง "หน่วงเวลา" การตอบกลับ (เช่น 2-3 วินาที) เพื่อลดความเร็วในการโจมตีของแฮกเกอร์
3. **Logout & Invalidation (The Blacklist):** เมื่อผู้ใช้สั่ง Logout แม้ Token จะยังไม่หมดอายุ (ยังไม่ถึง `exp`) แต่ต้องทำให้ Token นั้นใช้งานไม่ได้อีกต่อไป (ห้ามใช้แค่การลบ Cookie ฝั่ง Client)

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

🚀 Code Starter Template

1. **`utils/rateLimiter.js` (การจำกัดจำนวนครั้ง)**
2. **`controllers/authController.js` (Login Delay & Logout)**
3. **`middleware/authMiddleware.js` (Update Protect เพื่อเช็ค Blacklist)**

📋 คำแนะนำ (ไม่ใช่ Checklist)

1. **ดูที่ Response Time (สำหรับ Login Delay):**
   - ใน Postman มุมขวาล่างจะมีค่า **Time** * ถ้าใส่รหัสผ่านผิด แล้วเวลาขึ้นแค่ **50-100ms** = **ตก** (เพราะแฮกเกอร์ใช้โปรแกรมยิงรัวๆ ได้)
   - ถ้าขึ้น **2000ms+** = **ผ่าน** (คุณได้สร้าง "กับดักความหน่วง" ให้แฮกเกอร์แล้ว)
2. **ดูความศักดิ์สิทธิ์ของ Logout:**
   - นี่คือจุดตาย! ปกติคนมักแค่ลบ Cookie แต่ถ้าโจรแอบก๊อปค่า Token ไว้ โจรยังเข้าได้อยู่
   - ถ้าคุณ Logout แล้ว แต่ยังเอา Token เดิมมาเรียก API ได้ = **ตก**
   - ถ้า Logout แล้ว Token เดิม "ตายสนิท" ทันที = **ผ่าน**
3. **ดูการบล็อก IP:**
   - เมื่อลองยิงรัวๆ จนโดนบล็อก ให้ลองเปลี่ยนไป Login ด้วย User อื่น (ที่รหัสถูก) แต่ใช้ IP เดิม
   - ถ้าผ่าน = คุณต้อง **ยังเข้าไม่ได้** (เพราะโดนบล็อกที่ IP) นี่คือการกันระบบจากบอทสุ่มรหัสอย่างแท้จริง

Checklist Function 5.5

C1. การทดสอบการป้องกัน Brute Force (Brute Force Block Check)

เป้าหมาย: เพื่อดูว่าระบบสามารถหยุดยั้งโปรแกรมสแกนรหัสผ่าน (Script) ได้หรือไม่

- **วิธีทดสอบ:**
  1. ใน Postman ให้พยายามยิง Login ด้วยรหัสผ่านที่ผิดติดต่อกันรัวๆ
  2. **ถ้าผ่าน:** เมื่อถึงครั้งที่ 6 ระบบต้องไม่ตรวจสอบรหัสผ่านให้แล้ว แต่ต้องตอบกลับด้วย **429 Too Many Requests** พร้อมข้อความแจ้งเตือนให้รอ 15 นาที
- **จุดที่ต้องเช็ค:** ลองเปลี่ยนไปใช้อีเมลอื่น (แต่ IP เดิม) ในการ Login ครั้งที่ 6 ถ้ายังโดนบล็อกอยู่ แสดงว่า `rate-limiter` ทำงานถูกต้อง (บล็อกที่ต้นทาง IP ไม่ได้บล็อกที่บัญชี)

C2. การทดสอบความหน่วงเทียม (Artificial Delay Check)

เป้าหมาย: เพื่อลดประสิทธิภาพของแฮกเกอร์ในการเดารหัสผ่าน (ลดจำนวนครั้งที่แฮกเกอร์จะยิงได้ต่อวินาที)

- **วิธีทดสอบ:**
  1. ดูที่มุมขวาล่างของ Postman ในส่วนของ **Time** (เวลาที่ใช้ในการ Response)
  2. ลองใส่รหัสผ่านที่ **"ผิด"** แล้วกดส่ง
  3. **ถ้าผ่าน:** ค่า Time ต้องแสดงอย่างน้อย **2000 ms (2 วินาที)** ขึ้นไป
- **จุดที่ต้องระวัง:** หากใส่รหัสผ่าน **"ถูก"** ระบบควรจะตอบกลับเร็วตามปกติ (ไม่ต้องหน่วง) เพื่อไม่ให้กระทบประสบการณ์ใช้งานของผู้ใช้ทั่วไป

C3. การทดสอบ Logout ที่แท้จริง (Real Logout / Blacklist Check)

เป้าหมาย: ทำลายพลังของ Token ทันทีที่กด Logout (ต่อให้ Token ยังไม่หมดอายุก็ต้องใช้ไม่ได้)

- **วิธีทดสอบ:**
  1. ล็อกอินให้สำเร็จ แล้วก๊อปปี้ Token เก็บไว้
  2. กดเรียก API Logout (`/api/logout`)
  3. นำ Token ที่ก๊อปไว้ในข้อ 1 มาใส่ใน Header แล้วลองเรียก API ที่ติด `protect`
- **ถ้าผ่าน:** ระบบต้องตอบกลับ **401 Unauthorized** และบอกว่า Token นี้ใช้ไม่ได้แล้ว (Invalidated)
- **จุดที่ต้องเช็คในโค้ด:** ใน Middleware `protect` ต้องมีคำสั่งที่ไปเช็คใน `tokenBlacklist` ว่ามี Token นี้อยู่หรือไม่ ก่อนที่จะไปทำ `jwt.verify`

C4. การตรวจสอบการทำงานตาม IP (IP-Based Limit Check)

เป้าหมาย: มั่นใจว่าเราไม่ได้บล็อกคนทั้งบริษัทที่ใช้ Gateway เดียวกัน หรือบล็อกผิดคน

- **วิธีดูในโค้ด:** ตรวจสอบว่าไม่ได้เผลอไปใช้ฟิลด์ `email` เป็นตัวนับ (Key) ใน Rate Limiter แต่ให้ใช้ค่าเริ่มต้นของ Library ซึ่งก็คือ `IP Address`
- **ถ้าผ่าน:** คุณสามารถเปิด Tab ใหม่ใน Browser (หรือใช้เครื่องอื่นในวงแลนเดียวกัน) เพื่อทดสอบว่าโดนบล็อกพร้อมกันหรือไม่

C5. ตรรกะการตรวจสอบ Blacklist (Blacklist Logic Placement Check)

เป้าหมาย: ประสิทธิภาพและความปลอดภัย

- **วิธีดูในโค้ด:** ในไฟล์ `authMiddleware.js` ส่วนของฟังก์ชัน `protect`
- **ถ้าผ่าน:** การเช็ค `tokenBlacklist.includes(token)` ต้องเกิดขึ้น **"ก่อน"** การไป Query ฐานข้อมูล (`User.findById`) เพื่อประหยัดทรัพยากรเซิร์ฟเวอร์ (ถ้าตั๋วโดนแบนแล้ว ก็ไม่ต้องไปเสียเวลาหาข้อมูล User ใน DB)

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ